Computer Science and Engineering Department

NATIONAL INSTITUTE OF TECHNOLOGY

DELHI

Session (2025-2026)

Big Data Analytics

CSBB 422

'IoT Sensor Data Analysis in Real Time'

Submitted To:
Dr. Priten
(Assistant Professor)
Computer Science
Engineering Department

Submitted By:

Abdul Arsh	221210002
Aditya Shaurya Singh Negi	221210012
Akshat Singh	221210015
Gautham Binoy	221210039
Immanuel Thomas Francis	221210050

B. Tech CSE 7th Semester

Abstract

This project implements a fully containerized real-time streaming pipeline designed to ingest, buffer, and persist IoT sensor data efficiently. Sensor events generated from an Android device are first published over MQTT, then seamlessly bridged into Apache Kafka for scalable ingestion and temporary buffering. From Kafka, the data is periodically batched and written into HDFS using a date-partitioned directory structure to support organized long-term storage and downstream analytics.

A lightweight Flask-based frontend provides a near–real-time visualization layer, rendering live charts for accelerometer and gyroscope axes to help monitor device motion patterns. This document outlines the overall system architecture, explains each component in the deployment, and provides the essential commands and usage instructions needed to run and interact with the pipeline.

Acknowledgment

We sincerely acknowledge the invaluable guidance and support provided by the Computer Science and Engineering Department at the National Institute of Technology Delhi. Their academic environment and resources greatly contributed to the successful completion of this work. We extend our special thanks to **Dr. Priten (Assistant Professor)** for his insightful feedback, continuous encouragement, and constructive suggestions, all of which played a significant role throughout the development of this IoT sensor data analysis project.

Table Of Contents

1. Project Overview

We have developed a streamlined streaming pipeline designed to ingest, buffer, and store IoT sensor data. The system workflow is as follows:

- **Ingestion:**
  A mobile device (Android) publishes sensor data to a public MQTT broker (**broker.emqx.io**).
- **Streaming:**
  The script **m.py** (running in a Dockerized Python client) subscribes to MQTT topics and bridges the data into a Kafka topic (**sensors**).
- **Storage:**
  The script **k.py** consumes data from Kafka, buffers it in memory, and writes it to HDFS in batch intervals.
- **Infrastructure:**
  - **Hadoop:** A test cluster with **1 NameNode (master)** and **2 DataNodes (workers/slaves)** using BDE Hadoop images.
  - **Kafka:** Confluent Platform images with Zookeeper.
  - **Networking:** All services communicate via a shared Docker bridge network (**bigdata**).

Hadoop Master and Slave Nodes (NameNode & DataNodes)

To support distributed storage, Hadoop uses a **master–slave architecture**, which operates as follows:

- **NameNode (Master):**
  The NameNode is the central controller of HDFS. It does **not** store user data. Instead, it manages:
  • The directory structure of HDFS
  • Metadata (file names, permissions, timestamps)
  • Mapping of file blocks to DataNodes
  • Monitoring DataNode health via heartbeats

  Essentially, the NameNode acts as the **filesystem manager** and keeps the cluster organized and consistent.

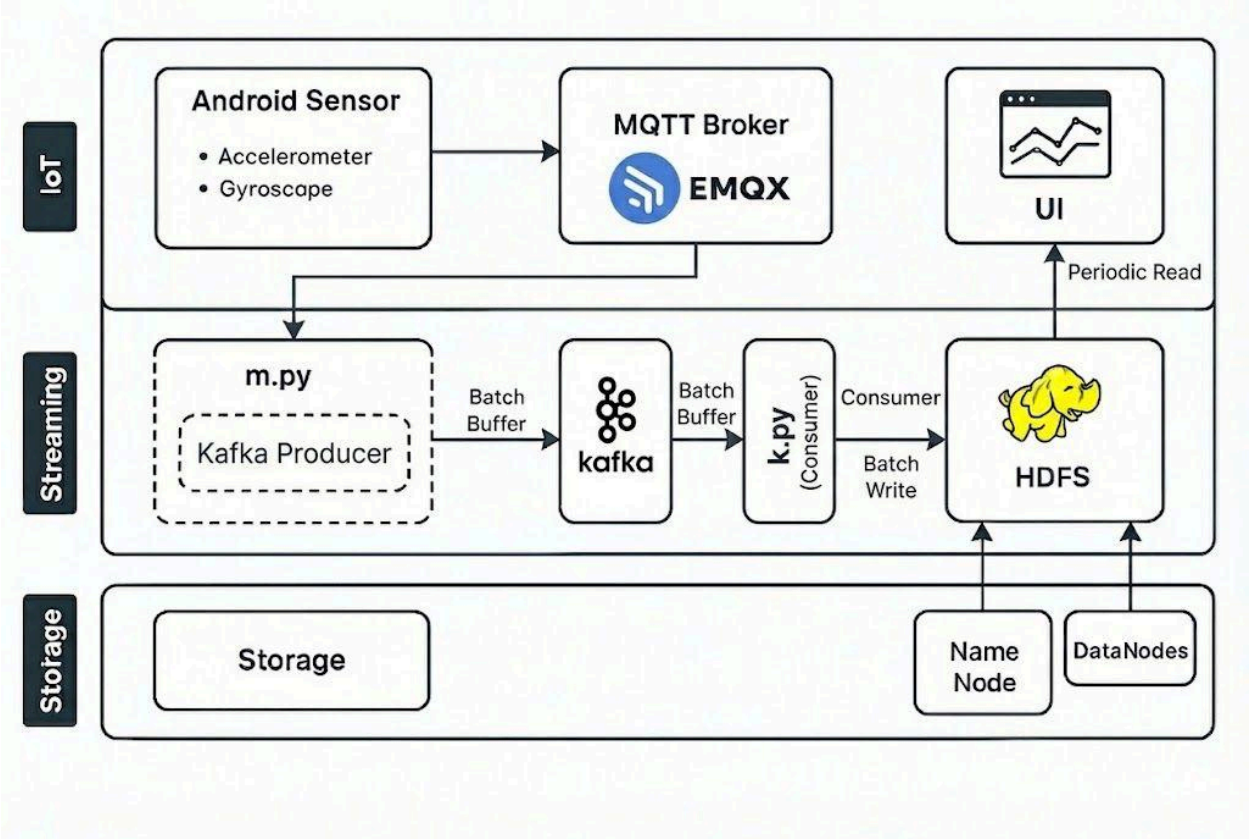
- **DataNodes (Slave/Worker Nodes):**
  DataNodes store the **actual data blocks** written into HDFS. Their responsibilities include:
  • Serving read/write requests
  • Storing blocks assigned by the NameNode
  • Sending heartbeats to prove they are alive
  • Replicating data for fault tolerance

  DataNodes act as the **storage layer** of the system.

- **How They Work Together:**
  When **k.py** writes data to HDFS:
  • The NameNode receives the write request and decides where blocks should be stored.
  • It assigns specific DataNodes and ensures data is replicated properly.
  • The DataNodes handle the actual writing and storage of the data.
  • This distributed model ensures **fault tolerance, scalability, and efficient data processing**.

2. Architecture (Logical)

The data flow moves from the edge (sensor) through a message broker, into the streaming platform, and finally into distributed storage.



Data Flow:

1. `[Android Sensor]` → **MQTT Broker** (Public)
2. **MQTT Broker** → `python-client:m.py`
3. `m.py` → **Kafka** (Docker Service)
4. **Kafka** → `python-client:k.py`
5. `k.py` → **HDFS** (NameNode + DataNodes)

Storage Schema: Files are stored in HDFS with the following directory structure to support time-series analysis: `/iot/sensors/date=YYYY-MM-DD/data.json`

3. Prerequisites

To replicate this environment, the following are required:

- Host OS: Windows 10/11 (with WSL2 recommended) or any modern Linux distribution (Ubuntu 22.04+, Fedora, etc.). macOS (Apple Silicon or Intel) also works if Docker Desktop is installed. Ensure virtualization is enabled in BIOS/firmware.
- Container Engine: Docker Desktop (Windows/macOS) or Docker Engine + Docker Compose plugin (Linux). Minimum version: Docker Engine 24.x; Compose plugin v2.20+ for stability with network aliases.
- CPU/RAM: At least 2 vCPUs and 4 GB RAM usable by Docker; recommended 4 vCPUs and 8 GB RAM when running Kafka + HDFS + UI concurrently. Adjust in Docker Desktop resources panel if containers are being OOM-killed.
- Disk Space: 2–3 GB free for base images (Kafka, Hadoop, Python). Additional space scales with ingested sensor data; consider pruning old images periodically.
- Git (Optional but Recommended): For cloning and version tracking (git clone workflow). Alternative: download ZIP archive from repository.
- Python: Not required on host (all execution uses python:3.10 container). Host Python may be used for quick prototyping but is intentionally decoupled from pipeline runtime.
- Network Connectivity: Stable outbound internet connection (pull images, connect to public MQTT broker). If behind a proxy, configure Docker daemon JSON or environment variables accordingly.

Port Mappings (Host → Container):

- `9870`: NameNode Web UI
- `9092`: Kafka Broker
- `8080`: Kafka UI

4. Files & Repository Layout

Suggested directory structure for the project:

```
bigdata/                # Project root
├── docker-compose.yml    # Service orchestration
├── scripts/              # Python scripts mounted into container
│   ├── m.py              # MQTT -> Kafka producer
│   └── k.py              # Kafka -> HDFS consumer
└── connect-plugins/      # (Optional) Local connector jars
```

Component Purpose:

- docker-compose.yml: Single orchestration file declaring all services, networks, and volumes for reproducible startup.
- scripts/m.py: Bridges MQTT telemetry into Kafka, translating lightweight sensor messages into durable log entries.
- scripts/k.py: Consumes Kafka topic sensors, batches events, and persists them into date-partitioned HDFS storage.

5. Docker Compose Configuration

The `docker-compose.yml` defines the service stack. Key configurations include:

- **Kafka Listeners:** configured as `PLAINTEXT://kafka:9092` to ensure internal container resolution works correctly.
- **Hadoop:** Uses BDE images with WebHDFS enabled on the NameNode.
- **Python Client:** A generic `python:3.10` container running `tail -f /dev/null` to keep it alive. The local `./scripts` folder is mounted as a volume.

6. Start / Stop Commands

Run the following commands from the project root:

Start the Stack:

- docker compose config
- docker compose pull
- docker compose up -d

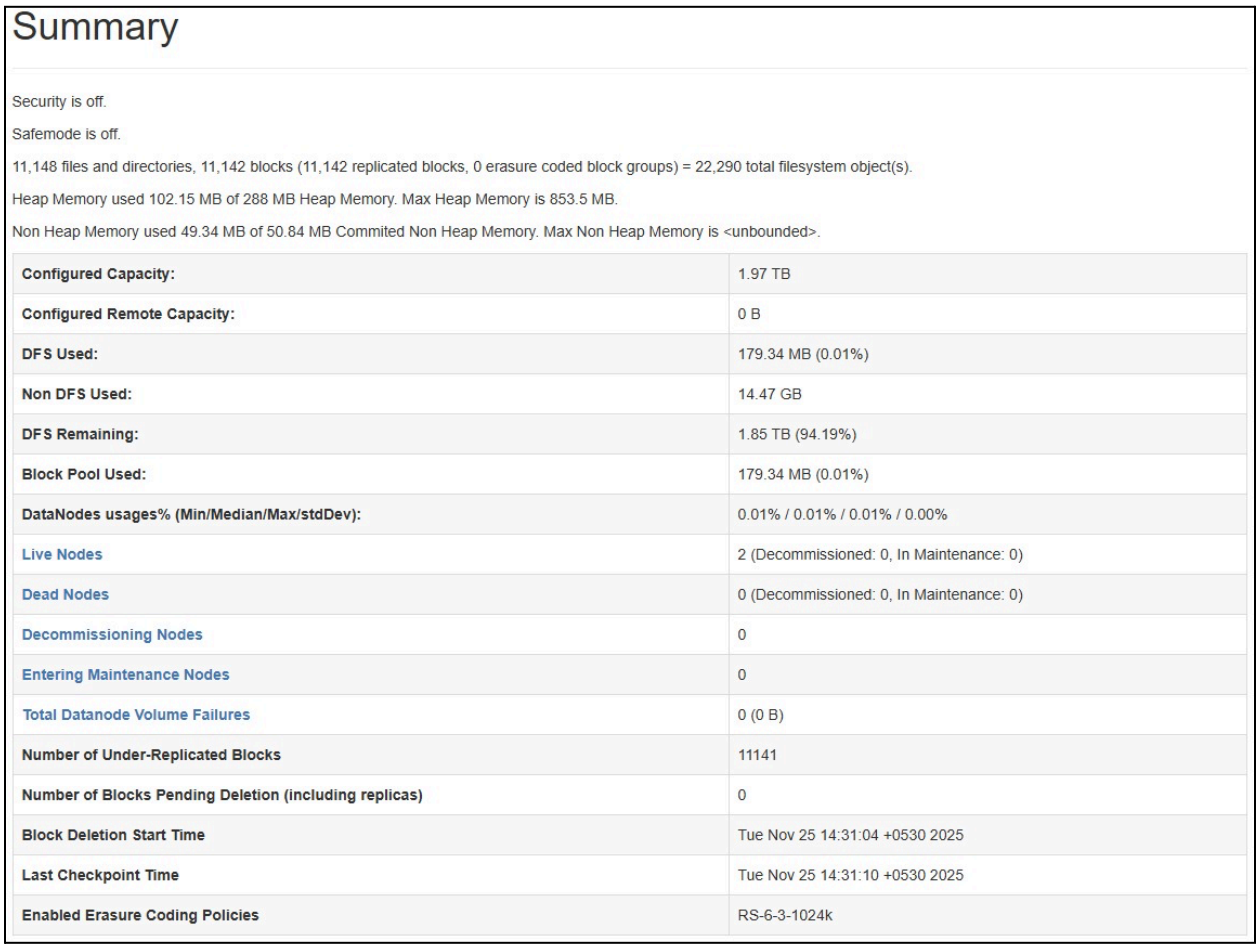
Stop the Stack:

- docker compose down

Restart only the python client:

- docker compose up -d python-client

7. Verifying Services & Web UIs



Summary

Security is off.

Safemode is off.

11,148 files and directories, 11,142 blocks (11,142 replicated blocks, 0 erasure coded block groups) = 22,290 total filesystem object(s).

Heap Memory used 102.15 MB of 288 MB Heap Memory. Max Heap Memory is 853.5 MB.

Non Heap Memory used 49.34 MB of 50.84 MB Commited Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	1.97 TB
Configured Remote Capacity:	0 B
DFS Used:	179.34 MB (0.01%)
Non DFS Used:	14.47 GB
DFS Remaining:	1.85 TB (94.19%)
Block Pool Used:	179.34 MB (0.01%)
DataNodes usages% (Min/Median/Max/stdDev):	0.01% / 0.01% / 0.01% / 0.00%
Live Nodes	2 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	11141
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Tue Nov 25 14:31:04 +0530 2025
Last Checkpoint Time	Tue Nov 25 14:31:10 +0530 2025
Enabled Erasure Coding Policies	RS-6-3-1024k

1. **Check Containers:** Run `docker ps`.
2. Ensure the following are active: `zookeeper`, `kafka`, `kafka-ui`, `namenode`, `datanode1`, `datanode2`, `python-client`.
3. **HDFS Explorer:** Visit `http://localhost:9870`
4. **Kafka UI:** Visit http://localhost:8080

8. Python Clients (Source Code)

The following scripts run inside the python-client container.

A. MQTT → Kafka Producer (m.py)

• Connects to the public MQTT broker and subscribes to Android sensor topics.
• Each received MQTT message is decoded and immediately forwarded to the Kafka topic **sensors**.
• Uses an asynchronous Kafka Producer to handle message delivery efficiently.
• Acts as the real-time ingestion layer, converting raw mobile sensor data into structured Kafka stream messages.

B. Kafka → HDFS Consumer (k.py)

• Subscribes to the Kafka topic **sensors** and continuously polls for new data.
• Buffers incoming messages in memory to avoid frequent writes and reduce load on HDFS.
• Periodically groups buffered data and writes it into HDFS as JSON.
• Stores files using date-based directories, enabling organized daily partitions inside Hadoop.
• Serves as the storage component that reliably persists Kafka streams into the distributed filesystem.

9. Execution Guide

To run the scripts within the Docker environment:

1. **Start the container:** `docker compose up -d python-client`
2. **Access the shell:** `docker exec -it python-client bash`
3. **Install dependencies:** `pip install hdfs confluent-kafka paho-mqtt`
4. **Run the scripts (in separate terminal windows):**
    - Terminal A: `python k.py` (Start this first)
    - Terminal B: `python m.py`

10. Frontend Web UI — IoT Sensor Visualizer

Summary
A lightweight Flask-based frontend visualizer was implemented to display recent sensor data stored in HDFS. The UI shows live/near-real-time charts for acceleration and gyroscope values (x, y, z), supports manual refresh and auto-refresh, and fetches only the required number of recent data-points from the backend.

Purpose & Behavior

1. The frontend provides two Chart.js charts:
    - Acceleration (x,y,z)
    - Gyroscope (x,y,z)
2. Controls available to the user:
    - **Samples:** number of sample points to request from the backend.
    - **Refresh Now:** force an immediate reload.
    - **Auto-refresh:** when enabled, the UI polls the backend periodically for new points.
3. Data flow:
    - The Flask backend reads the latest partitioned files under `/iot/sensors` in HDFS (daily `data.json` files or newline JSON records).
    - Endpoints convert those HDFS files into a chronological list of sensor messages and return only the requested number of samples.
    - The frontend requests `/api/data` or `/api/latest_file_points` and renders charts using Chart.js.

Notes on HDFS integration

- The Flask backend uses `InsecureClient('http://namenode:9870', user='root')` to connect to WebHDFS (the same HDFS client used in the pipeline). Because WebHDFS replies with DataNode redirects (container hostnames), **run the Flask app inside the same Docker Compose network** (e.g., the `python-client` container) so container hostnames resolve correctly. See `app.py` for the exact client configuration.
- The backend gracefully handles both:
    - **Daily partition files:** `/iot/sensors/date=YYYY-MM-DD/data.json` (JSON array), and
    - **Line-delimited logs:** NDJSON or individual messages written to files.
- Endpoints parse timestamps and normalize accelerometer/gyro vectors to `{x, y, z}` for consistent charting.

Frontend implementation details (index.html & app.js)

- **`index.html`** provides the structure, chart canvas elements, control inputs, and includes Chart.js.
- **`app.js`** (client script):
    - Calls `/api/data?count=<n>` (or `/api/latest_file_points`) to fetch a JSON array of normalized points.
    - Builds two Chart.js line charts (acceleration and gyroscope) and updates them on refresh.
    - Handles auto-refresh timers, sample size change, and manual refresh requests.

Output: